

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Arini Yanua Sari - 5024241015

2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Menyiapkan alat dan bahan, yaitu board STM32 Blackpill, ST-Link V2, kabel USB, dan laptop yang telah terpasang STM32CubeIDE.
2. Menghubungkan ST-Link V2 ke board STM32 Blackpill melalui pin SWDIO, SWCLK, GND, dan 3.3V, kemudian menghubungkan ST-Link ke laptop.
3. Membuat proyek baru pada STM32CubeIDE dan memilih mikrokontroler yang sesuai dengan board Blackpill.
4. Mengonfigurasi pin LED onboard (misalnya PC13) sebagai *GPIO Output* pada *Pinout & Configuration*.
5. Mengatur sumber *clock* dan mode *debug* (Serial Wire) pada konfigurasi sistem.
6. Menghasilkan kode program (*Generate Code*), lalu menambahkan fungsi `HAL_GPIO_TogglePin()` dan `HAL_Delay()` pada fungsi `while(1)` di `main.c`.
7. Melakukan proses *build* untuk memastikan tidak ada *error*.
8. Mengunggah program ke board (*Run/Flash*).
9. Mengamati LED onboard dan mencatat hasilnya, yaitu LED berkedip sesuai interval *delay* yang diberikan.

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Menyiapkan alat dan bahan, yaitu board ESP32, kabel USB, LED, resistor, *breadboard*, dan laptop yang telah terpasang ESP-IDF.
2. Menghubungkan board ESP32 ke laptop dan memastikan *port* COM terdeteksi.
3. Membuat proyek baru ESP-IDF (misalnya dari contoh *blink*) dan memilih *target* chip yang sesuai.
4. Menentukan pin GPIO untuk LED, lalu mengonfigurasinya sebagai *output* menggunakan `gpio_set_directi`
5. Menulis program yang mengatur kondisi pin secara bergantian dengan `gpio_set_level()` dan jeda waktu `vTaskDelay()`.
6. Melakukan proses *build* proyek.
7. Melakukan *flash* program ke ESP32 melalui *port* yang sesuai.
8. Menjalankan *monitor* serial untuk memantau keluaran dan mengamati LED yang berkedip.

1.3 Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

1. Menyiapkan alat dan bahan, yaitu board ESP32, *push button*, resistor eksternal, LED, *breadboard*, dan kabel jumper.
2. Merangkai rangkaian *pull-up* eksternal: menghubungkan salah satu kaki tombol ke GPIO input, kaki lainnya ke GND, serta resistor dari GPIO input ke 3.3V.
3. Menulis program pada Arduino IDE dengan mengatur pin tombol sebagai `INPUT` dan membaca status tombol menggunakan `digitalRead()`.
4. Mengunggah program, menekan tombol, dan mencatat keadaan logika yang terbaca pada *Serial Monitor*.
5. Mengubah rangkaian menjadi *pull-down* eksternal dengan memindahkan resistor ke GND dan tombol ke 3.3V, lalu mengulangi pengamatan.
6. Menghapus resistor eksternal, kemudian mengubah mode pin menjadi `INPUT_PULLUP` untuk menguji *pull-up* internal.
7. Mengunggah program dan mengamati keadaan logika saat tombol ditekan maupun dilepas.
8. Membandingkan hasil ketiga konfigurasi (eksternal dan internal) dan mencatatnya dalam tabel pengamatan.

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

1. Menyiapkan alat dan bahan, yaitu board ESP32, *push button*, LED, resistor, *breadboard*, dan kabel jumper.
2. Merangkai tombol pada GPIO input dengan konfigurasi `INPUT_PULLUP` dan LED pada GPIO output.
3. Menulis program tanpa *debouncing* yang menghitung jumlah penekanan tombol dan menampilkannya pada *Serial Monitor*.
4. Mengunggah program, menekan tombol beberapa kali, dan mencatat jumlah hitungan yang terbaca (hitungan bisa bertambah lebih dari satu akibat *bouncing*).
5. Memodifikasi program dengan menambahkan *debouncing* berbasis perangkat lunak, yaitu menggunakan `millis()` untuk memeriksa selisih waktu antar perubahan status tombol (misalnya 50 ms).
6. Mengunggah program yang telah dimodifikasi dan mengulangi penekanan tombol dengan jumlah yang sama.
7. Membandingkan hasil sebelum dan sesudah *debouncing*, lalu mencatatnya pada tabel pengamatan.

2 Analisis Hasil Percobaan

Bagian ini membahas hasil yang diperoleh selama praktikum Modul 1 dengan membandingkannya terhadap teori, serta mengidentifikasi faktor-faktor yang memengaruhi hasil percobaan.

2.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Pada percobaan ini, LED onboard pada board STM32 Blackpill berhasil berkedip sesuai interval *delay* yang diprogram. Hal ini menunjukkan bahwa proses konfigurasi pin sebagai *GPIO Output*, pembuatan kode melalui STM32CubeIDE, serta proses *flashing* menggunakan ST-Link berjalan dengan benar. Fungsi `HAL_GPIO_TogglePin()` bekerja dengan membalik kondisi logika pin pada setiap pemanggilannya, sedangkan `HAL_Delay()` menahan eksekusi program selama waktu yang ditentukan sehingga LED terlihat menyala dan padam secara bergantian. Hasil ini sesuai dengan teori bahwa keluaran GPIO dapat dikendalikan melalui perubahan nilai register keluaran. Perlu dicatat bahwa LED onboard pada Blackpill bersifat *active low*, sehingga LED menyala ketika pin berlogika rendah (0). Kesalahan umum pada percobaan ini adalah koneksi ST-Link yang kurang baik atau pemilihan konfigurasi *debug* yang keliru, yang menyebabkan program gagal diunggah.

2.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Pada percobaan ini, LED yang terhubung ke ESP32 berhasil berkedip menggunakan framework ESP-IDF. Berbeda dengan Percobaan 1, ESP-IDF berbasis FreeRTOS sehingga jeda waktu dibuat menggunakan `vTaskDelay()`, yang tidak memblokir seluruh sistem melainkan hanya menunda *task* yang bersangkutan dan memberi kesempatan *task* lain berjalan. Hasil percobaan sesuai dengan teori, yaitu pin GPIO harus dikonfigurasi terlebih dahulu sebagai *output* sebelum dapat dikendalikan melalui `gpio_set_level()`. Faktor yang dapat memengaruhi hasil antara lain pemilihan *port* COM yang salah, pemilihan *target* chip yang tidak sesuai, serta kesalahan pada instalasi atau pengaturan *environment* ESP-IDF. Dibandingkan dengan STM32, ESP-IDF memiliki alur kerja yang lebih panjang (konfigurasi, *build*, *flash*, dan *monitor*), namun menyediakan kontrol yang lebih lengkap terhadap sistem.

2.3 Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

Percobaan ini menunjukkan bahwa pin input yang tidak diberi referensi tegangan akan berada pada kondisi *floating*, sehingga nilai logika yang terbaca tidak stabil. Dengan resistor *pull-up*, pin berlogika HIGH saat tombol tidak ditekan dan berubah menjadi LOW saat tombol ditekan. Pada konfigurasi *pull-down*, logikanya berlawanan, yaitu LOW saat tombol dilepas dan HIGH saat ditekan. Penggunaan `INPUT_PULLUP` memberikan hasil yang sama dengan *pull-up* eksternal tanpa memerlukan komponen tambahan, karena ESP32 memiliki resistor *pull-up* internal pada pin GPIO-nya. Hasil ini sesuai dengan teori. Perbedaannya, resistor internal memiliki nilai yang relatif besar (sekitar puluhan kΩ) dan tidak dapat diubah, sedangkan resistor eksternal dapat dipilih sesuai kebutuhan rangkaian. Kesalahan yang dapat terjadi antara lain kesalahan penyambungan resistor atau kaki tombol, serta penggunaan pin yang tidak mendukung *pull-up* internal (misalnya GPIO 34 sampai 39 pada ESP32 yang hanya berfungsi sebagai *input only* tanpa resistor internal).

2.4 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

Pada program tanpa *debouncing*, satu kali penekanan tombol dapat terbaca lebih dari satu kali sehingga jumlah hitungan bertambah melebihi jumlah penekanan sebenarnya. Hal ini disebabkan oleh fenomena *bouncing*, yaitu kontak mekanis tombol yang memantul beberapa kali dalam hitungan milidetik sebelum mencapai kondisi stabil, dan mikrokontroler yang cepat membaca setiap pantulan tersebut sebagai penekanan terpisah. Setelah ditambahkan *debouncing* berbasis perangkat lunak menggunakan `millis()` dengan jeda sekitar 50 ms, hitungan menjadi sesuai dengan jumlah penekanan yang sebenarnya. Hasil ini sesuai dengan teori bahwa perubahan status hanya diterima jika sinyal telah stabil selama waktu tertentu. Faktor yang memengaruhi hasil adalah kualitas tombol, karena setiap tombol memiliki karakteristik *bounce* yang berbeda, serta nilai waktu *debounce* yang dipilih. Waktu yang terlalu singkat tidak menghilangkan *bounce* sepenuhnya, sedangkan waktu yang terlalu lama dapat membuat penekanan cepat tidak terbaca.

2.5 Percobaan 5 yang Tidak Dilaksanakan

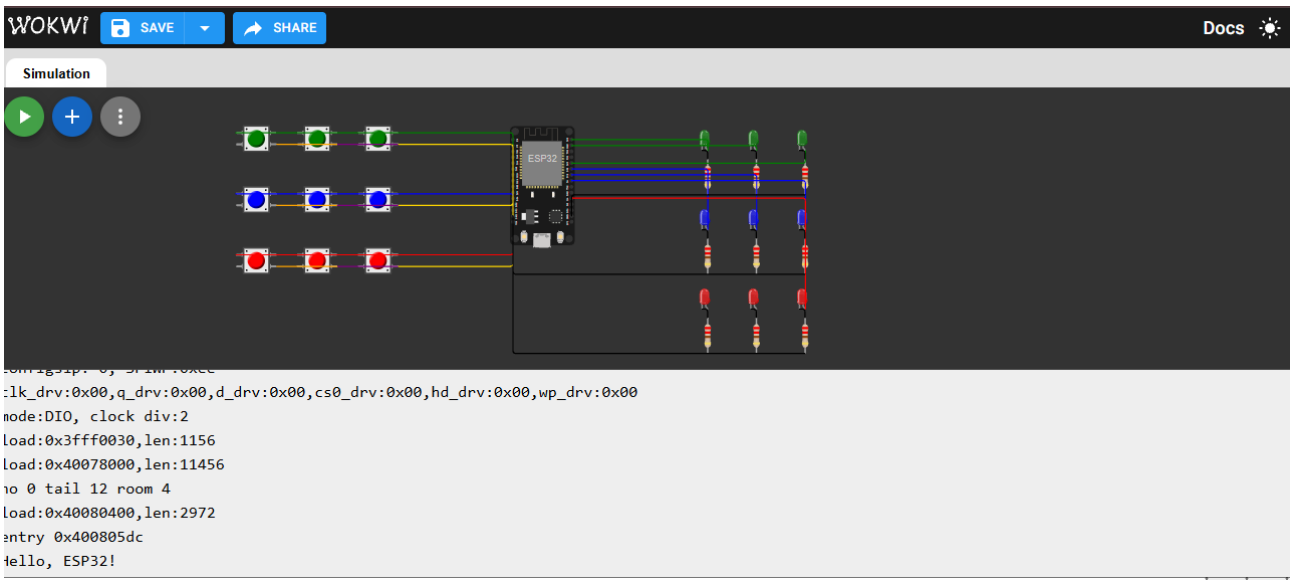
Percobaan 5 mengenai akses GPIO analog, yaitu pembacaan ADC dengan LDR menggunakan ESP32 dan framework Arduino, tidak dapat dilaksanakan. Hal ini dikarenakan waktu praktikum telah habis sebelum praktikan sempat menyelesaikan percobaan tersebut. Waktu yang tersedia banyak terpakai pada tahap persiapan dan pengerjaan percobaan sebelumnya, seperti instalasi dan konfigurasi *environment* serta penanganan kendala teknis pada percobaan awal. Akibatnya, hasil dan analisis untuk Percobaan 5 tidak dapat disajikan dalam laporan ini, dan pemahaman mengenai pembacaan sinyal analog melalui ADC belum dapat dievaluasi secara langsung melalui praktik.

3 Hasil Tugas Modul

Bagian ini memuat hasil pengerjaan tugas modul berupa simulasi Matrix Pushbutton 3x3 yang mengendalikan Matrix LED 3x3 menggunakan ESP32 pada platform Wokwi, beserta jawaban pertanyaan analisis.

3.1 Rancangan dan Hasil Simulasi

Rangkaian terdiri dari satu board ESP32, sembilan *push button* yang tersusun dalam matriks 3x3, dan sembilan LED yang juga tersusun 3x3 dengan resistor pembatas arus pada masing-masing LED. Setiap tombol dihubungkan ke satu pin GPIO input dan GND, serta dikonfigurasi dengan *pull-up* internal (`INPUT_PULLUP`). Setiap LED dihubungkan ke satu pin GPIO output melalui resistor. Tombol pada posisi (baris, kolom) tertentu mengendalikan LED pada posisi yang sama. Pada setiap tombol diterapkan *debouncing* berbasis `millis()` sebagaimana Percobaan 4, sehingga status LED hanya berubah setelah pembacaan tombol stabil selama 50 ms.



Gambar 1: Rangkaian dan hasil simulasi Matrix Pushbutton 3x3 mengendalikan Matrix LED 3x3 pada Wokwi

Source code yang digunakan ditunjukkan sebagai berikut.

```
const int NUM = 9;
const int btnPin[NUM] = {13, 12, 14, 27, 26, 25, 33, 32, 15};
const int ledPin[NUM] = {23, 22, 21, 19, 18, 5, 17, 16, 4};

const unsigned long DEBOUNCE_MS = 50;

bool ledState[NUM];
int lastReading[NUM];
int stableState[NUM];
unsigned long lastChange[NUM];

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < NUM; i++) {
    pinMode(btnPin[i], INPUT_PULLUP);
    pinMode(ledPin[i], OUTPUT);
    digitalWrite(ledPin[i], LOW);
    ledState[i] = false;
    lastReading[i] = HIGH;
    stableState[i] = HIGH;
    lastChange[i] = 0;
  }
}

void loop() {
  for (int i = 0; i < NUM; i++) {
    int reading = digitalRead(btnPin[i]);

    // Jika pembacaan berubah, catat waktunya
    if (reading != lastReading[i]) {
      lastChange[i] = millis();
```

```
32        lastReading[i] = reading;
33      }
34
35      // Terima perubahan hanya jika sudah stabil selama DEBOUNCE_MS
36      if ((millis() - lastChange[i]) > DEBOUNCE_MS &&
37          reading != stableState[i]) {
38        stableState[i] = reading;
39
40        // Tombol ditekan (LOW karena pull-up): toggle LED
41        if (stableState[i] == LOW) {
42          ledState[i] = !ledState[i];
43          digitalWrite(ledPin[i], ledState[i]);
44          Serial.printf("Tombol baris %d kolom %d -> LED %s\n",
45                        i / 3 + 1, i % 3 + 1,
46                        ledState[i] ? "ON" : "OFF");
47        }
48      }
49    }
50  }
```

Pada simulasi, setiap penekanan tombol mengubah status LED pada posisi yang sesuai (menyala atau padam) tanpa terjadi perubahan ganda akibat *bouncing*.

3.2 Analisis Pertanyaan

3.2.1 Pertanyaan 1: Alasan Debouncing Tetap Diperlukan

Bouncing bukan disebabkan oleh banyaknya tombol yang ditekan bersamaan, melainkan oleh sifat mekanis setiap tombol. Di dalam *push button* terdapat kontak logam yang bergerak. Saat ditekan atau dilepas, kontak tersebut tidak langsung menempel stabil, tetapi memantul beberapa kali dalam hitungan milidetik sehingga menghasilkan rentetan perubahan HIGH–LOW sebelum mencapai kondisi stabil. Karena itu, walaupun hanya satu tombol yang ditekan pada satu waktu, tombol tersebut tetap mengalami *bouncing*. Mikrokontroler ESP32 membaca pin dalam orde mikrodetik, jauh lebih cepat daripada durasi pantulan, sehingga setiap pantulan terbaca sebagai penekanan terpisah. Pada rangkaian ini, di mana tombol men-*toggle* LED, satu kali penekanan dapat memicu *toggle* berkali-kali dan membuat status akhir LED tidak sesuai harapan (misalnya LED tampak tidak bereaksi saat jumlah pantulan genap). Dengan *debouncing*, perubahan hanya diterima jika sinyal stabil selama waktu tertentu, sehingga satu penekanan menghasilkan satu perubahan status.

3.2.2 Pertanyaan 2: Konfigurasi Pull-up pada Pembacaan Tombol

Rangkaian ini menggunakan konfigurasi *pull-up* internal (`INPUT_PULLUP`), dengan satu kaki tombol ke pin GPIO dan kaki lainnya ke GND. Saat tombol tidak ditekan, pin tersambung ke 3.3V melalui resistor *pull-up* internal sehingga berlogika **HIGH**. Saat tombol ditekan, pin terhubung langsung ke GND sehingga berlogika **LOW**. Pin input tidak boleh dibiarkan *floating* karena pin tersebut berada pada kondisi impedansi tinggi dan tidak terhubung ke tegangan referensi mana pun. Pin yang *floating* mudah terpengaruh *noise* dan interferensi elektromagnetik, bahkan medan dari tubuh atau kabel di

sekitarnya, sehingga nilainya acak dan tidak dapat diprediksi. Akibatnya, mikrokontroler dapat membaca tombol seolah-olah ditekan padahal tidak, dan LED berubah status sendiri tanpa penekanan. Resistor *pull-up* memastikan pin selalu memiliki kondisi logika yang pasti (HIGH) saat tombol tidak ditekan.

3.2.3 Pertanyaan 3: Digital Input vs Analog Input (ADC)

Pada pembacaan digital, pin GPIO hanya membedakan dua kondisi, yaitu HIGH dan LOW, berdasarkan ambang tegangan (*threshold*). Tegangan di bawah ambang dibaca sebagai LOW dan di atas ambang dibaca sebagai HIGH, sehingga informasi yang diperoleh hanya berupa dua keadaan, seperti tombol ditekan atau tidak. Pada pembacaan analog, pin dihubungkan ke *Analog-to-Digital Converter* (ADC) yang mengukur besar tegangan sebenarnya pada pin dan mengubahnya menjadi nilai bilangan bulat. ADC pada ESP32 memiliki resolusi 12 bit, sehingga tegangan 0–3.3V dipetakan ke 4096 tingkat nilai (0–4095), dengan resolusi sekitar 0.8 mV per tingkat. Karena itu ADC dapat merepresentasikan banyak kondisi, bukan hanya dua. Jika tombol diganti dengan LDR yang dirangkai sebagai pembagi tegangan, perubahan intensitas cahaya mengubah resistansi LDR dan tegangan pada pin secara kontinu, sehingga nilai ADC yang terbaca bervariasi sesuai tingkat terang atau gelap. Dalam penggunaannya, pin yang dipakai harus mendukung ADC (misalnya GPIO 32–39 pada ADC1), dan hasilnya dapat dibandingkan dengan nilai ambang tertentu untuk menentukan kapan LED dinyalakan.

4 Kesimpulan

Berdasarkan hasil praktikum Modul 1 mengenai pengendalian GPIO pada mikrokontroler STM32 dan ESP32, diperoleh kesimpulan sebagai berikut.

1. Pengendalian GPIO sebagai *output* berhasil dilakukan pada dua platform berbeda. Pada STM32 Blackpill, LED berkedip menggunakan STM32CubeIDE dengan fungsi `HAL_GPIO_TogglePin()` dan `HAL_Delay()`, sedangkan pada ESP32 dengan framework ESP-IDF, LED berkedip menggunakan `gpio_set_level()` dan `vTaskDelay()`. Kedua platform memiliki alur kerja dan pendekatan pemrograman yang berbeda, namun prinsip pengendalian pin *output*-nya sama.

2. Pin GPIO *input* tidak boleh dibiarkan dalam kondisi *floating* karena nilai logikanya menjadi tidak stabil. Penggunaan resistor *pull-up* maupun *pull-down*, baik eksternal maupun internal, memastikan pin memiliki kondisi logika yang pasti. Hasil percobaan sesuai dengan teori, yaitu *pull-up* menghasilkan logika HIGH saat tombol dilepas dan LOW saat ditekan, sedangkan *pull-down* sebaliknya. Resistor internal (`INPUT_PULLUP`) memberikan hasil yang setara dengan resistor eksternal tanpa komponen tambahan.

3. *Bouncing* pada tombol mekanis menyebabkan satu kali penekanan terbaca lebih dari satu kali. Teknik *debouncing* berbasis perangkat lunak menggunakan `millis()` efektif mengatasi masalah ini sehingga satu penekanan menghasilkan satu perubahan status.

4. Pada tugas modul, simulasi Matrix Pushbutton 3x3 yang mengendalikan Matrix LED 3x3 pada Wokwi berhasil dibuat dengan menerapkan *pull-up* internal dan *debouncing* pada setiap tombol, sehingga LED hanya berubah status setelah pembacaan tombol stabil. Hal ini menunjukkan bahwa konsep GPIO *input*, *output*, *pull-up*, dan *debouncing* dapat digabungkan dalam satu sistem yang bekerja dengan baik.

5. Percobaan 5 mengenai akses GPIO analog (pembacaan ADC dengan LDR) tidak dapat dilaksanakan karena waktu praktikum habis. Oleh karena itu, pemahaman mengenai pembacaan sinyal analog baru diperoleh secara teori, dan belum dibuktikan melalui praktik langsung.

Secara keseluruhan, seluruh percobaan yang dilaksanakan (Percobaan 1 sampai 4) dan tugas modul memberikan hasil yang sesuai dengan teori. Praktikum ini memberikan pembelajaran mengenai dasar pengendalian GPIO, pentingnya pemberian referensi tegangan pada pin *input*, serta perlunya penanganan *bouncing* untuk mendapatkan pembacaan tombol yang andal. Sebagai tindak lanjut, Percobaan 5 sebaiknya dipelajari dan dicoba secara mandiri di luar jam praktikum.

5 Lampiran

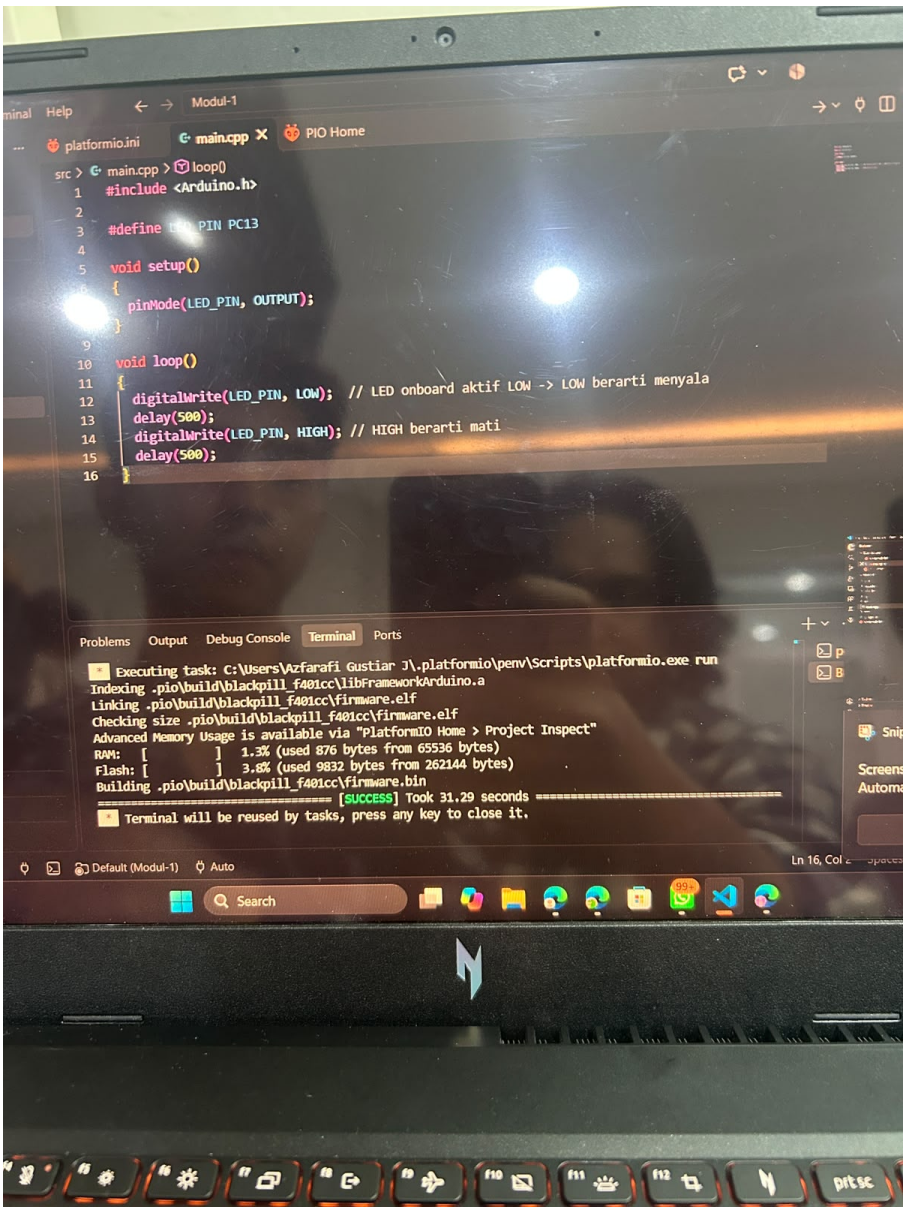
5.1 Dokumentasi saat praktikum

Menampilkan foto selama pelaksanaan praktikum. Dokumentasi meliputi foto alat yang digunakan dan foto praktikan saat praktikum. Tujuannya sebagai bukti telah dilakukan kegiatan praktikum.

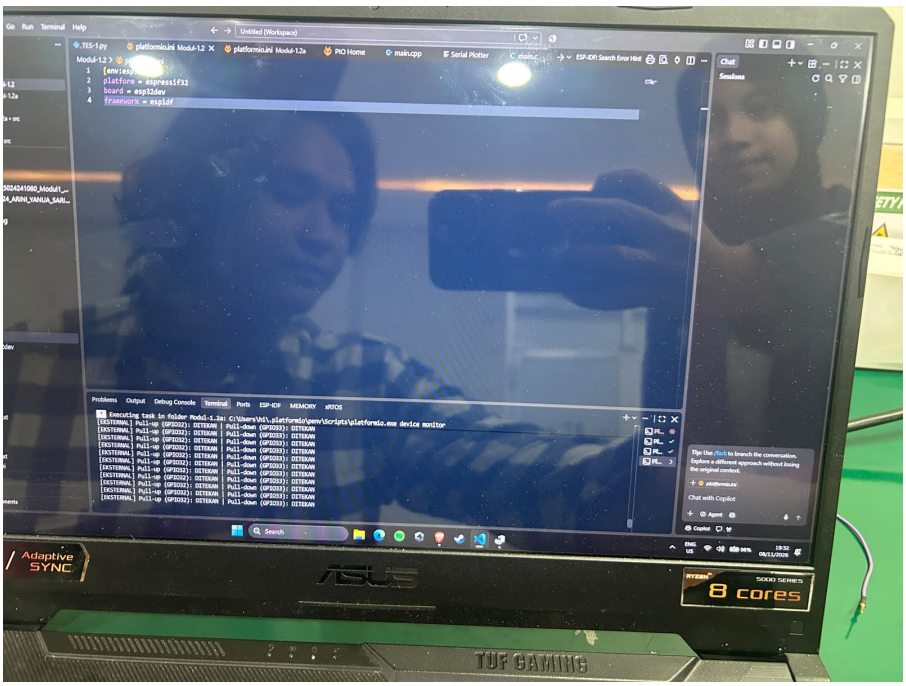
5.1.1 Foto Alat yang Digunakan



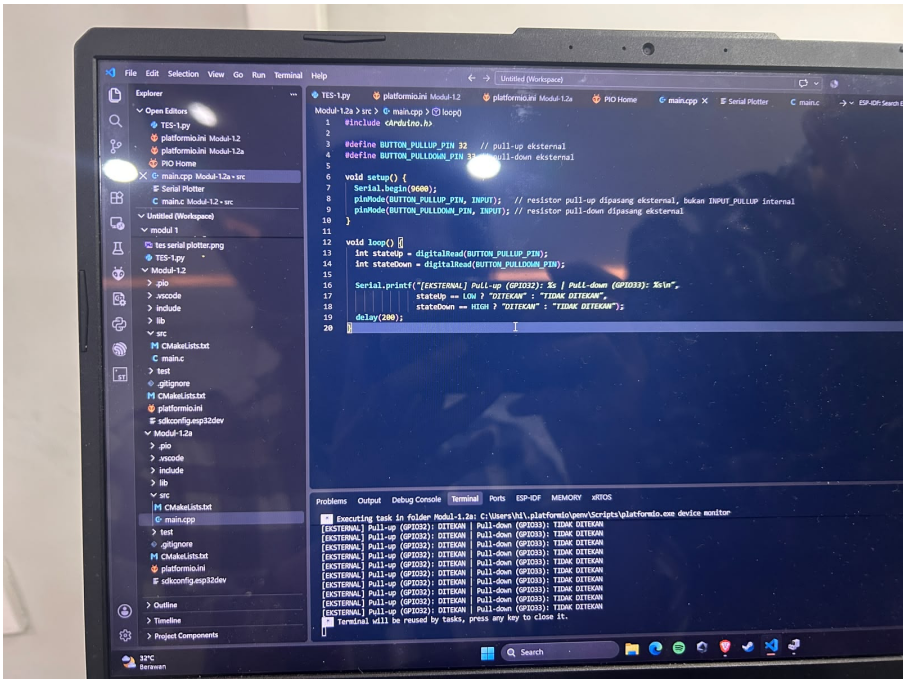
Gambar 2: foto Komponen



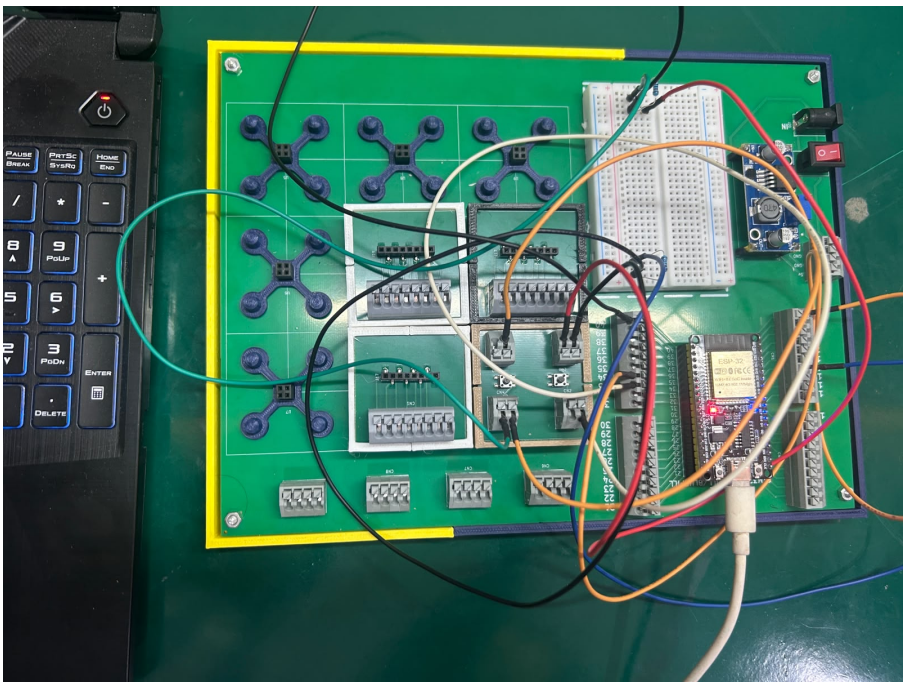
Gambar 3: percobaan 1



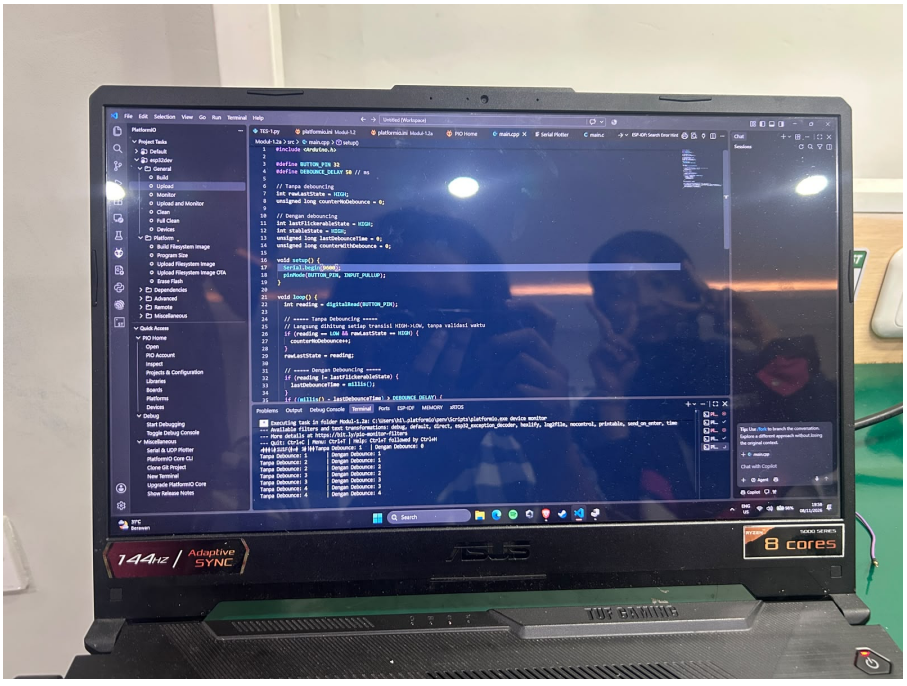
Gambar 4: percobaan 2



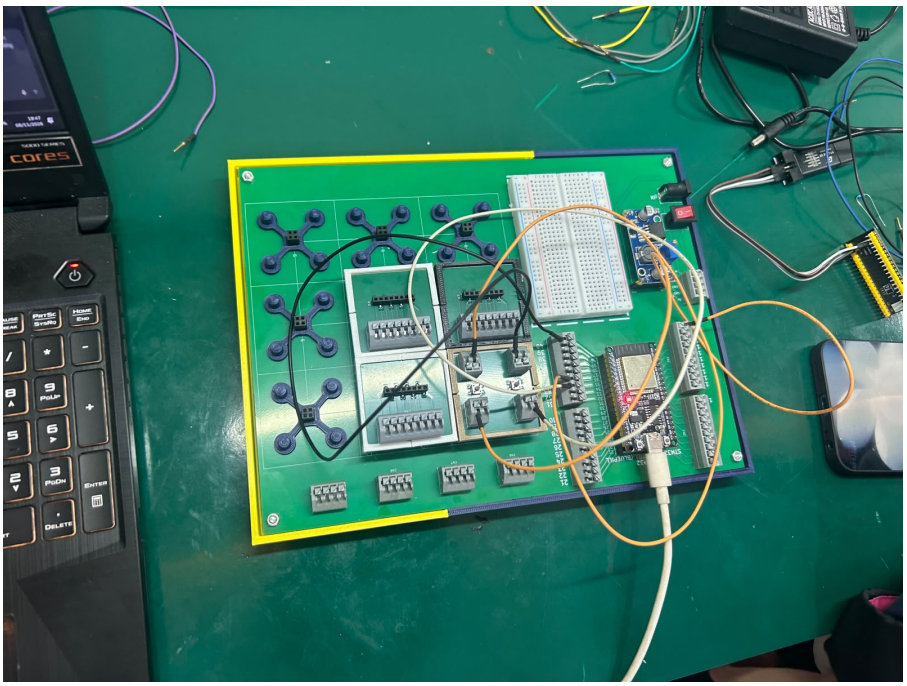
Gambar 5: percobaan 3



Gambar 6: Rangkaian Percobaan 3



Gambar 7: percobaan 4



Gambar 8: Rangkaian Percobaan 4

5.1.2 Foto Praktikan saat Praktikum



Gambar 9: foto praktikan)